



Reconocimiento de Patrones Estadísticos

Proyecto Final

June 8, 2026

Kaleb Alejandro Aguilar Ávila

Maestría en Ciencias de la Computación y Matemáticas Industriales

kaleb.aguilar@cimat.mx

Resumen

Este reporte presenta la implementación del artículo *Simplifying Random Forests' Probabilistic Forecasts* sobre datos abiertos del sistema de bicicleta pública EcoBici Guadalajara (febrero 2026). El objetivo es predecir la duración de los viajes en minutos aplicando el método Top- k , que simplifica la distribución de pronóstico del Random Forest reteniendo únicamente los k vecinos de mayor peso. La implementación se desarrolló en Python, en el notebook `topk_ecobici.ipynb`.

1 Introducción

Los Random Forests son modelos ampliamente utilizados en tareas de regresión y clasificación. La predicción del RF puede expresarse como una combinación lineal ponderada de todas las observaciones del conjunto de entrenamiento. Esta representación permite la construcción directa de distribuciones de pronóstico conocidas como *Quantile Regression Forests* (QRF), sin necesidad de reentrenar el modelo.

La predicción del RF para una nueva observación x_0 se escribe como:

$$\hat{\mu}_{\text{RF}}(x_0) = \sum_{i=1}^n w_i(x_0) y_i, \quad (1)$$

donde los pesos $w_i(x_0)$ se obtienen promediando los pesos de hoja de cada árbol individual:

$$w_i(x_0) = \frac{1}{B} \sum_{b=1}^B w_i(x_0, \xi_b), \quad w_i(x_0, \xi_b) = \frac{\mathbf{1}[x_i \in R_{\ell(x_0, \xi_b)}]}{\sum_{j=1}^n \mathbf{1}[x_j \in R_{\ell(x_0, \xi_b)}]}. \quad (2)$$

El mismo vector de pesos permite estimar Y dado que $X = x_0$ como:

$$\hat{F}(t | x_0) = \sum_{i=1}^n w_i(x_0) \mathbf{1}(y_i \leq t). \quad (3)$$

El método propuesto en el artículo, denominado **Top- k** , consiste en retener únicamente los k pesos más grandes para cada observación de prueba y renormalizarlos. Sea $\mathcal{I}_k(x_0)$ el conjunto de índices de los k pesos de mayor valor; los pesos simplificados quedan:

$$\tilde{w}_i(x_0) = \begin{cases} \frac{w_i(x_0)}{\sum_{j \in \mathcal{I}_k(x_0)} w_j(x_0)} & \text{si } i \in \mathcal{I}_k(x_0), \\ 0 & \text{en otro caso.} \end{cases} \quad (4)$$

Para $k = n$ se recupera el RF completo; valores pequeños de k producen distribuciones de pronóstico más interpretables, representables como un conjunto reducido de escenarios con probabilidades asociadas.

2 Metodología

2.1 Datos y preprocesamiento

El conjunto de datos contiene los viajes del sistema EcoBici de Guadalajara durante febrero de 2026. Tras eliminar registros con valores faltantes se obtuvieron 331 315 entradas; luego de filtrar duraciones no positivas y superiores a $\min(p_{99}, 240)$ minutos quedaron $n = 326\,162$ registros.

La variable objetivo es la duración del viaje en minutos, calculada directamente a partir de las marcas temporales de inicio y fin. La duración se acotó con el percentil 99 de la distribución y un límite absoluto de 240 minutos. Ambos criterios se aplican como filtro mediante $\min(p_{99}, 240)$.

A partir de las marcas temporales se derivaron cuatro variables extra: hora de inicio, día de la semana (0–6), e indicador binario de fin de semana. Estas variables codifican la ciclicidad de la demanda sin requerir transformaciones trigonométricas adicionales, dado que los modelos de árbol explotan particiones ordinales de forma nativa. La edad del usuario se estimó como diferencia entre el año actual y el año de nacimiento declarado, asumiendo que el cumpleaños ya ocurrió; se trata de una aproximación de ± 1 año aceptable.

Se excluyeron del conjunto de características los identificadores de viaje y usuario, las marcas temporales originales y el año de nacimiento (redundante con la edad). Mientras que la variable respuesta es $y =$ duración del viaje en minutos. Dejando los siguientes predictores: **Genero** (0/1), **edad**, **hora_inicio**, **dia_semana**, **es_fin_semana**, **Origen_Id** y **Destino_Id**. La partición se realizó en orden cronológico (70 %/30 %), con $n_{\text{train}} = 228\,313$ y $n_{\text{test}} = 97\,849$. En la Figura 1, se muestra como a pesar partir los datos por cronología la partición no un sesgo en la duración del viaje.

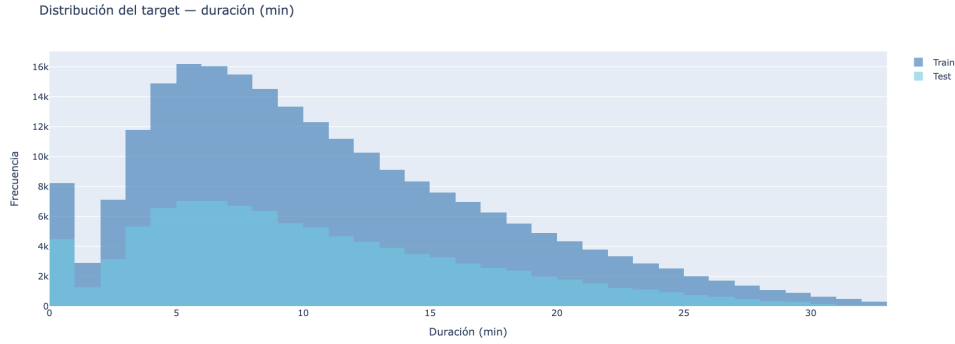


Figura 1: Distribuciones de duración del viaje para ambos datasets (Train, Test).

2.2 Construcción del Random Forest

El RF se construyó desde cero mediante $B = 1000$ instancias de `DecisionTreeRegressor`, cada una entrenada sobre una muestra bootstrap con $\tilde{p} = \lfloor \sqrt{p} \rfloor$ features aleatorios por nodo, sin restricción de profundidad (Algoritmo 1).

Algorithm 1: Entrenamiento del Random Forest

Input: $(X_{\text{train}}, y_{\text{train}})$, B , $\tilde{p} = \lfloor \sqrt{p} \rfloor$

Output: $\{(T_b, \phi_b)\}_{b=1}^B$

for $b \leftarrow 1$ **to** B **do**

$\text{idx}_b \leftarrow$ muestra bootstrap de tamaño n_{train} con reemplazo;
 $\phi_b \leftarrow$ subconjunto aleatorio de $\tilde{p}$ columnas;
 Entrenar T_b sobre $(X_{\text{train}}[\text{idx}_b], \phi_b, y_{\text{train}}[\text{idx}_b])$;
 Almacenar (T_b, ϕ_b) ;

Cada árbol recibe su propio subconjunto de features ϕ_b , seleccionado sin reemplazo en cada nodo raíz y mantenido fijo durante todo el crecimiento del árbol. Esta decisión difiere de la convención habitual de *sklearn*, que reselecciona features en cada nodo interno; aquí el muestreo ocurre una sola vez por árbol, esto hace que cada árbol sea un regresor sobre un subespacio fijo de dimensión $\tilde{p}$.

Se optó por no imponer restricción de profundidad (`max_depth=None`), permitiendo que cada árbol crezca hasta hojas puras. La semilla de cada árbol se fijó como `random_state=b`.

La predicción del ensamble se obtiene como el promedio muestral de las B predicciones individuales:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}_{\phi_b}). \quad (5)$$

La evaluación se realizó sobre el conjunto de prueba retenido, reportando MAE, RMSE y R^2 como métricas de desempeño.

2.3 Cálculo de la matriz de pesos

Por costo computacional, los pesos se calcularon para un subconjunto $N_{\text{demo}} = 300$ observaciones de prueba, obteniendo la matriz $W \in \mathbb{R}^{300 \times 228313}$ mediante co-ocurrencia de hojas (Algoritmo 2), tal como indica la ecuación (2).

Cada entrada W_{ij} acumula, a través de los B árboles, la fracción de veces que la observación de prueba i y la observación de entrenamiento j caen en la misma hoja. La normalización por el tamaño de la hoja $s_i = |\{j : \ell_j^{\text{tr}} = \ell_i^{\text{de}}\}|$ garantiza que cada árbol contribuya con pesos que suman la unidad para cada punto de consulta, independientemente de cuán poblada esté la hoja. La guarda $s[s=0] \leftarrow 1$ previene divisiones por cero en hojas que no contienen ningún punto de entrenamiento; en la práctica este caso es infrecuente pero posible cuando $\tilde{p}$ es pequeño.

Algorithm 2: Cálculo de la matriz de pesos por co-ocurrencia de hojas

Input: $X_{\text{train}}, X_{\text{demo}}, \{(T_b, \phi_b)\}_{b=1}^B$
Output: $W \in \mathbb{R}^{N_{\text{demo}} \times n_{\text{train}}}$
 $W \leftarrow \mathbf{0}$;
for *cada* (T_b, ϕ_b) **do**
 $\ell^{\text{tr}} \leftarrow T_b.\text{APPLY}(X_{\text{train}}[:, \phi_b]);$
 $\ell^{\text{de}} \leftarrow T_b.\text{APPLY}(X_{\text{demo}}[:, \phi_b]);$
 $M \leftarrow (\ell^{\text{de}}_{[:, \text{None}} = \ell^{\text{tr}}_{\text{None}, :})$ // **booleana** $N_{\text{demo}} \times n_{\text{train}}$
 $s \leftarrow M.\text{sum}(\text{axis}=1)$; $s[s=0] \leftarrow 1$;
 $W += M / s$;
 $W \leftarrow W/B$;

2.4 Simplificación Top- k

A partir de W se aplica la ecuación (4) para $k \in \{3, 5, 10\}$ (Algoritmo 3). Antes de seleccionar los k vecinos se anulan las entradas con peso inferior a $\varepsilon = 10^{-5}$, correspondientes a co-ocurrencias en muy pocos árboles. El algoritmo aplica dos normalizaciones: la primera iguala la escala entre filas antes de la selección; la segunda convierte los pesos retenidos en una distribución discreta sobre los k vecinos, de modo que $\hat{y}^{(k)} = \tilde{W} y_{\text{train}}$ sea una esperanza condicional.

Algorithm 3: Predicción Top- k

Input: W, y_{train}, k
Output: $\hat{y}^{(k)}, \tilde{W}$
 $\tilde{W} \leftarrow W.\text{copy}()$;
 $\tilde{W}[\tilde{W} < 10^{-5}] \leftarrow 0$;
 $\tilde{W} \leftarrow \tilde{W} / \tilde{W}.\text{sum}(\text{axis}=1)$ // **pre-normalización**
for $j \leftarrow 1$ **to** N_{demo} **do**
 $\mathcal{I}_k(x_j) \leftarrow \text{ARGTOP-}k(\tilde{W}[j, :])$;
 $\tilde{W}[j, \ell] \leftarrow 0$ para todo $\ell \notin \mathcal{I}_k(x_j)$;
 $\tilde{W} \leftarrow \tilde{W} / \tilde{W}.\text{sum}(\text{axis}=1)$ // **renormalización final**
 $\hat{y}^{(k)} \leftarrow \tilde{W} y_{\text{train}}$;

3 Resultados

La Tabla 1 resume el desempeño sobre los $N_{\text{demo}} = 500$ casos de prueba evaluados.

Table 1: Métricas de RF completo vs. Top- k .

Modelo	MAE (min)	RMSE (min)	R^2
RF completo	5.073	6.212	0.1097
Top-3	2.756	4.498	0.5333
Top-5	2.774	4.390	0.5554
Top-10	2.994	4.505	0.5317

Las tres variantes Top- k superan al RF completo en las tres métricas. La reducción de MAE respecto al RF completo es de aproximadamente 45 % para Top-5, lo que sugiere que concentrar los pesos en los vecinos más similares elimina el ruido de co-ocurrencias débiles que el promedio completo arrastra.

Top-5 obtiene el mejor desempeño global: MAE de 2.774 min, RMSE de 4.390 min y $R^2 = 0.555$. Top-3 alcanza un MAE prácticamente idéntico (2.756 min) pero con RMSE y R^2 ligeramente inferiores, indicando que un vecindario de tres observaciones puede ser demasiado local en algunos casos. Top-10 muestra un deterioro moderado respecto a Top-5, lo que apunta a que incorporar vecinos adicionales más allá de cinco introduce observaciones marginalmente menos representativas sin beneficio apreciable.

El R^2 del RF completo (0.110) es notablemente bajo considerando que se entrena sobre el conjunto completo; esto es consistente con el sesgo hacia la media que introduce el promedio sobre n_{train} ponderaciones, muchas de ellas cercanas a cero pero no nulas. La Figura 2 ilustra este efecto: el RF completo concentra las predicciones en una banda estrecha alrededor de 10 minutos.

Igualmente en la Figura 3 la distribución de residuos. Aquí es evidente que el método Top-K generó residuos más consistentes centrados en cero.

Predicciones vs real — RF completo y Top- k

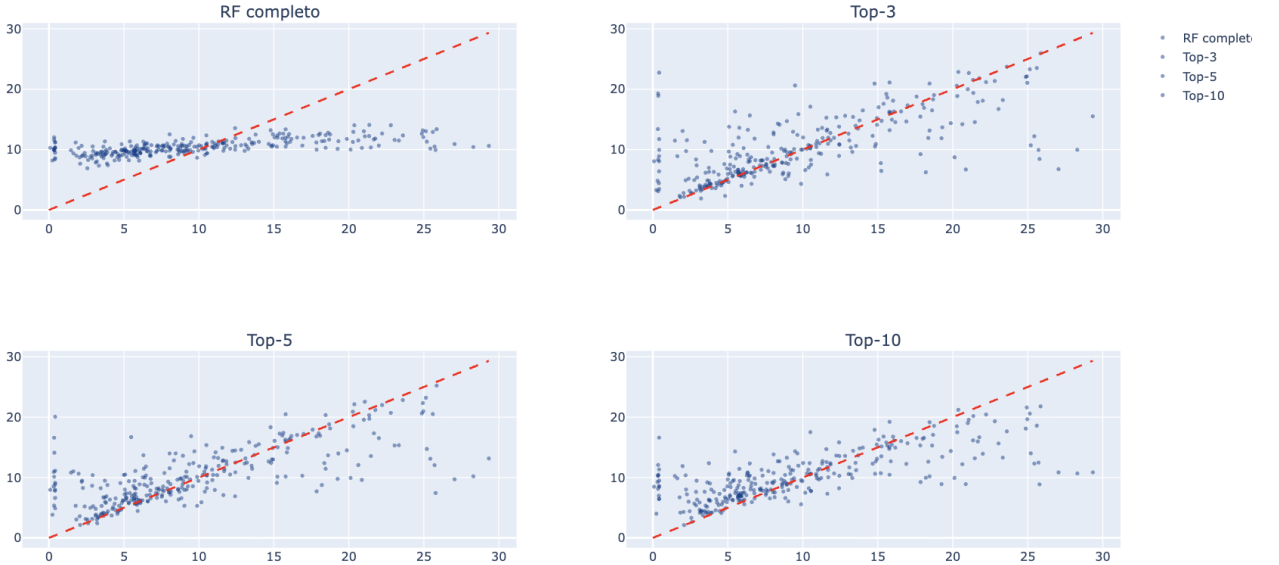


Figura 2: Comparativa Tiempo de viaje vs Predicción

independientemente del valor real, mientras que los modelos Top- k recuperan la dispersión de la diagonal.

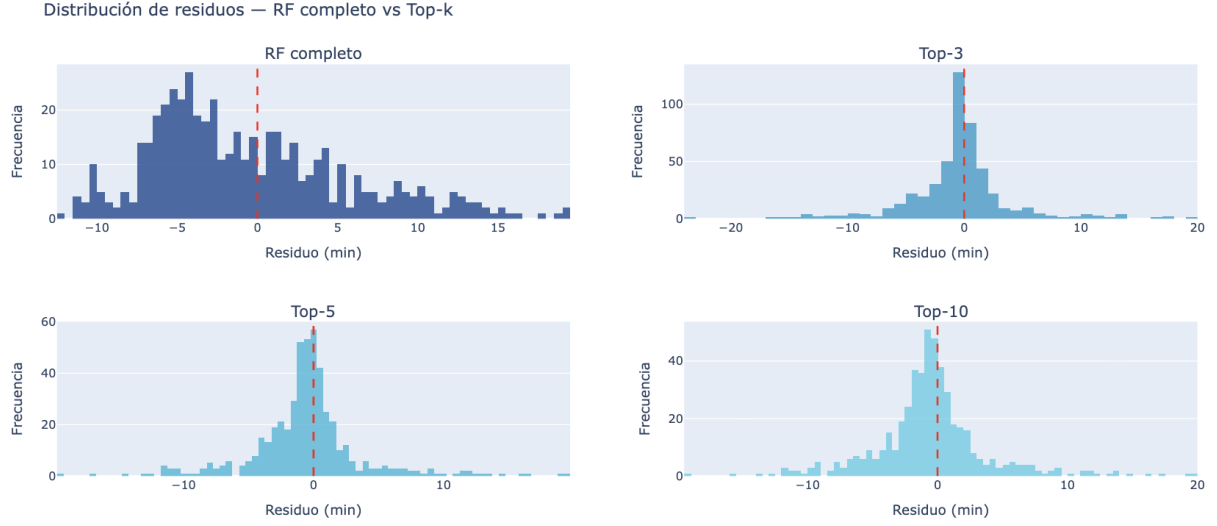


Figura 3: Distribución de residuos en la predicción

4 Conclusiones

La implementación del método Top- k sobre los datos de EcoBici Guadalajara confirma el hallazgo central del artículo de referencia: la simplificación de la distribución de pronóstico del RF, al retener únicamente los k vecinos de mayor peso, no solo preserva sino que mejora el desempeño predictivo. Con $k = 5$ se obtiene una reducción del 45 % en MAE respecto al RF completo y un R^2 de 0.555, frente al 0.110.

Este resultado es consistente con lo descrito en el artículo: cuando los pesos pequeños son estimados con ruido, descartarlos constituye una forma de regularización más acertada. En el contexto de viajes urbanos en bicicleta, la duración depende fuertemente de un vecindario reducido de viajes similares en origen, destino y hora, por lo que la concentración de masa de probabilidad en k observaciones representativas resulta adecuada.